PIM IV — ETAPA 6 · MODELAGEM UML (EXECUÇÃO EXTERNA)

Manual de Orientação — Criação dos Diagramas UML

CodeRyse Academy / Sistema Acadêmico Integrado

Classes | Sequência | Casos de Uso | Componentes

Documento: Manual_Orientacao_Diagramas_UML.pdf

Data de elaboração: 23 de outubro de 2026

Natureza do documento: roteiro textual de execução — não contém nenhum diagrama, imagem, código Mermaid/PlantUML ou arquivo de diagrama pronto

Ferramenta de destino: à escolha do desenvolvedor (Astah, Visual Paradigm, StarUML, Enterprise Architect, diagrams.net, ou equivalente)

> ⚠ **Este documento é exclusivamente um guia de instruções. Nenhum diagrama foi desenhado, gerado ou incorporado durante a produção deste manual — essa é uma responsabilidade do desenvolvedor, a ser executada posteriormente na ferramenta externa escolhida, seguindo as instruções abaixo.**

Sumário

1. Orientações gerais
2. Guia para o diagrama de classes
3. Guia para os diagramas de sequência
4. Guia para o diagrama de casos de uso
5. Outros diagramas a avaliar
6. Matriz de rastreabilidade
7. Procedimento de execução pelo desenvolvedor
8. Glossário de notação UML (textual)

1. Orientações gerais

1.1 Objetivo de cada tipo de diagrama neste projeto

Diagrama	O que ele precisa responder
Classes	Como o domínio é modelado em C# — entidades, herança, atributos, relacionamentos e multiplicidades reais do EF Core.
Sequência	Como uma interação específica do usuário percorre Controller → Service → Repository/DbContext → banco, passo a passo.
Casos de uso	O que cada papel (Aluno/Professor/Coordenador/Admin/Visitante) pode fazer no sistema, comprovado por autorização real no código.
Componentes / Implantação (opcionais)	Como Web/Mobile/Desktop se conectam ao mesmo backend — "arquitetura em alinhamento", pedida pela Etapa 6.

1.2 Onde buscar informação (mapa geral de arquivos)

Informação	Onde está
Entidades de domínio	`Models/*.cs` (29 arquivos)
Relacionamentos, multiplicidades, cascade	`Data/PlataformaContext.cs` + `Data/Configurations/*.cs` (Fluent API — é a fonte de verdade de multiplicidade, mais confiável que inferir só pela entidade)
Endpoints e contratos de entrada/saída	`Controllers/*.cs` + `DTOs/*.cs`
Regras de negócio / fluxos	`Services/*.cs`
Contratos entre camadas	`Interfaces/*.cs`
Histórico de evolução do schema	`Migrations/*.cs` (ordem cronológica pelo nome do arquivo, formato `AAAAMMDDHHmmss_Nome.cs`)
Autorização por papel (base dos casos de uso)	Atributos `[Authorize(Roles="...")]` em cada Controller, mais checagens programáticas dentro dos Services (ex.: `PodeAcessarAluno`, `PodeGerenciarCursoAsync`)

1.3 Como identificar relacionamentos reais (não supor)

- Uma propriedade de navegação (`public Curso Curso { get; set; }` ou `public List<Modulo> Modulos { get; set; }`) indica associação — mas a **multiplicidade e o comportamento de exclusão** (Cascade/Restrict/NoAction) só são confiáveis olhando o arquivo de `Configurations/` correspondente, não o Model isoladamente.
- Uma FK com tipo nulável (`int? CursoId`) indica associação opcional (multiplicidade `0..1`); sem `?`, é obrigatória (`1`).
- Um índice único composto (`HasIndex(...).IsUnique()`) frequentemente expressa uma regra de negócio importante (ex.: "um aluno não pode ter duas matrículas aprovadas no mesmo curso") — vale anotar como restrição no diagrama ou em nota de rodapé, não só como relacionamento estrutural.

1.4 Como distinguir implementação atual de comportamento desejado

> Este projeto tem um caso real e documentado dessa distinção: a entidade `Turma` e a propriedade `Matricula.TurmaId` continuam existindo no banco, mas **não são mais usadas** pelo fluxo de matrícula vigente (ver seção 3, fluxo "Matrícula"). Se você representar `Turma` no diagrama de classes como se ainda fosse o mecanismo central de matrícula, estará desenhando um comportamento **histórico/pretérito**, não o atual. Regra geral: sempre que o código tiver uma propriedade, método ou tabela que pareça não ser mais chamada pelo fluxo principal, isso deve ser investigado (buscar todas as referências no código, não só a declaração) antes de decidir se ela entra no diagrama como ativa, como legada/observação, ou se fica de fora.

1.5 Como validar a fidelidade do diagrama ao código

1. Para cada classe desenhada, confirme que todo atributo e método existe literalmente no arquivo `.cs` correspondente (não adicionar "getters/setters óbvios" que não estão explícitos no código, se o projeto usa propriedades automáticas — adicioná-los como se fossem métodos é uma distorção).
2. Para cada relacionamento, confirme a multiplicidade no arquivo de `Configurations/`, não só na presença da propriedade de navegação.
3. Para cada caso de uso, confirme que existe um endpoint real e acessível por aquele ator (atributo `[Authorize]` correspondente) — não incluir funcionalidade que só existe na interface sem um endpoint de backend por trás, nem vice-versa, sem marcar a diferença.

1.6 Dúvidas que precisam ser esclarecidas antes de desenhar

- A tabela `Turma` deve aparecer no diagrama de classes como entidade ativa, como entidade legada (com nota), ou ser omitida? (recomendação: incluir com uma nota explícita de transição, dado que ela ainda existe fisicamente e é referenciada por FKs obrigatórias em `ConteudoDidatico` / `Avaliacao`).
- O diagrama de classes deve ser um único diagrama monolítico ou dividido por módulo (ver 2.7 — recomendação é dividir)?
- Os diagramas de sequência devem cobrir também os fluxos de IA (geração de questões) e de Avaliação formal, ou só os 7 fluxos centrais listados na seção 3? (este manual cobre os 7 centrais; fluxos de IA podem ser adicionados depois, seguindo o mesmo método).

2. Guia para o diagrama de classes

2.1 Projetos e pastas a examinar

`Models/` (29 arquivos), `Data/PlataformaContext.cs`, `Data/Configurations/` (7 arquivos: `AcademicoConfiguration.cs`, `AvaliacoesConfiguration.cs`, `ConteudosConfiguration.cs`, `NotificacaoConfiguration.cs`, `ProgressoConfiguration.cs`, `RefreshTokenConfiguration.cs`, `UsuariosConfiguration.cs`).

2.2 Como identificar classes de domínio vs. DTOs vs. controllers vs. serviços/repositórios

Categoria	Como reconhecer	Entra no diagrama de classes de domínio?
Entidade de domínio	Classe em `Models/`, mapeada no `DbSet<T>` do `PlataformaContext`	Sim — é o foco principal
DTO	Classe em `DTOs/`, usada como parâmetro/retorno de ação de Controller	Não no diagrama de domínio; se útil, pode virar um diagrama de classes separado "camada de contrato de API"
Controller	Classe em `Controllers/`, herda de `ControllerBase`, decorada com `[ApiController]`	Só no diagrama de pacotes/componentes (seção 5), não no de classes de domínio
Service / Repository	Classe em `Services/` ou `Repositories/`, implementa uma interface de `Interfaces/`	Idem — representar como componente, não como classe de domínio

2.3 Como verificar herança e implementação de interface

Herança de classe: procure `: NomeDaClasseBase` na declaração (ex.: `public class Aluno : Usuario`). Confirmado neste projeto para a hierarquia de usuários — ver 2.5. Implementação de interface: procure `: INomeDaInterface` nos Services/Repositories (ex.: `public class CursoService : ICursoService`) — represente como relação de realização (seta tracejada com ponta vazada) no diagrama de componentes/pacotes, não no diagrama de classes de domínio.

2.4 Como identificar associações e dependências

Associação: propriedade de navegação de um lado (ou dos dois lados) mais uma FK. Dependência (menos forte que associação): uma classe que recebe outra só como parâmetro de método, sem guardar referência permanente — menos comum entre entidades de domínio deste projeto, mais comum entre Services.

2.5 Hierarquia de usuários — como representar

`Usuario` é abstrata (arquivo `Models/Usuario.cs`), mapeada na tabela base `Usuarios` com as colunas comuns (id, nome, email, cpf, telefone, endereço completo, tipoUsuario, dataCadastro, senhaHash, ativo, dados de recuperação de senha). As 4 subclasses (`Aluno`, `Professor`, `Coordenador`, `Admin`) herdam dela e são mapeadas cada uma em sua própria tabela (`Alunos`, `Professores`, `Coordenadores`, `Admins`) — isto é o padrão **Table Per Type**, confirmado em `Data/Configurations/UsuariosConfiguration.cs` pelas chamadas `.ToTable(...)`. No diagrama, represente como generalização clássica (seta vazada apontando para `Usuario`) a partir de cada subtipo. Atributos extras por subtipo: `Aluno` → `Matricula` (string); `Professor` → `CodigoRegistro`, `Especialidade`; `Coordenador` → `CodigoRegistro`; `Admin` → nenhum atributo extra. Consulte os arquivos `Models/Usuario.cs`, `Aluno.cs`, `Professor.cs`, `Coordenador.cs`, `Admin.cs` para a lista exata de métodos públicos de cada um antes de desenhar.

2.6 Como determinar multiplicidades (via EF Core)

- FK não-nula (`int CursoId`) + navegação obrigatória → multiplicidade `1` do lado "muitos".
- FK nula (`int? TurmaId`) → multiplicidade `0..1`.
- Propriedade de coleção (`List<Modulo>`) sem FK correspondente na própria classe → lado `0..*` ou `1..*`, dependendo se a existência do pai é condição para o filho existir (ver `DeleteBehavior.Cascade` nas Configurations — Cascade costuma indicar composição, ou seja, o filho não existe sem o pai).
- Índice único composto por duas FKs (ex.: `(AlunoId, CursoId)`) indica uma regra de cardinalidade de negócio (não necessariamente uma multiplicidade UML pura) — documente como nota/restrição textual junto ao relacionamento.

2.7 Como dividir os diagramas por módulo (recomendado, para evitar excesso de informação)

Módulo sugerido	Classes centrais
Usuários	`Usuario`, `Aluno`, `Professor`, `Coordenador`, `Admin`, `RefreshToken`, `Notificacao`, `FeedbackAcademico`
Cursos e estrutura pedagógica	`Curso`, `ProfessorCurso`, `Modulo`, `Turma`, `ConteudoDidatico`, `Matricula`, `Pagamento`
Sistema de avaliação	`QuestaoBanco`, `AlternativaQuestaoBanco`, `AnexoQuestaoBanco`, `Avaliacao`, `QuestaoPublicada`, `AlternativaQuestaoPublicada`, `TentativaAvaliacao`, `RespostaAluno`, `LancamentoNotaAluno`
Progresso do aluno	`ProgressoConteudoAluno`, `ProgressoModuloAluno`, `ProgressoCursoAluno`

Para classes que aparecem em mais de um módulo (ex.: `Matricula` conecta Cursos e Avaliação via `TentativaAvaliacao`), represente-a nos dois diagramas, mas detalhe atributos/métodos completos só no diagrama do módulo "dono" (sugerido: Cursos).

2.8 Elementos a omitir

Omita do diagrama de classes de domínio: DTOs, Controllers, Services, Repositories, Interfaces, classes de configuração do EF Core, classes de middleware/Common, e classes de infraestrutura de teste (`TestContextFactory`). Esses pertencem ao diagrama de componentes/pacotes (seção 5), não ao de classes de domínio.

2.9 Como registrar dúvidas sobre relacionamento não comprovado

Se ao ler o código um relacionamento parecer ambíguo (ex.: uma FK sem navegação explícita, ou uma propriedade comentada como obsoleta), **não desenhe por suposição**. Anote a dúvida na coluna "Pendências de validação" da matriz de rastreabilidade (seção 6) com a referência exata de arquivo:linha, e decida com quem revisar o trabalho antes de finalizar o diagrama.

3. Guia para os diagramas de sequência

Para cada um dos 7 fluxos abaixo, a investigação no código já foi feita nesta auditoria. Use a descrição textual numerada como roteiro para desenhar a sequência na ferramenta externa: cada número vira uma mensagem/seta entre objetos (ex.: Ator → Controller → Service → DbContext → Banco → retorno).

3.1 Inicialização e carregamento de dados no Desktop

Arquivos a consultar: `desktop/main.js` (completo, 86 linhas).

1. `app.whenReady()` dispara a configuração de CSP e chama `criarJanelaPrincipal()`.
2. `BrowserWindow` é criado com `contextIsolation` / `sandbox` ativos.
3. `janela.loadURL(APP_URL)` — `APP_URL` resolvida de `DESKTOP_APP_URL` (env) ou, por padrão, a URL de produção do backend.
4. O servidor .NET responde com o HTML/JS/CSS da SPA (mesmo fluxo de qualquer requisição Web — ver `Program.cs:311-312,380`, `UseStaticFiles` + `MapFallbackToFile`).
5. A partir daí, o carregamento de dados (login, cursos etc.) segue os fluxos 3.2 em diante — executado pelo JavaScript da SPA, não pelo processo principal do Electron.

3.2 Login

Arquivos: `Controllers/AuthController.cs:20-38`, `Services/AuthService.cs:27-45,191-253`.

1. Cliente (SPA) → `POST /api/v1/Auth/login` com `LoginDto`.
2. `AuthController.Login` delega para `AuthService.LoginAsync`.
3. `AuthService` consulta `Usuario` por e-mail na tabela base (`PlataformaContext.Usuarios`).
4. Verifica senha com `BCrypt.Verify`; se inválida ou usuário inativo, retorna erro (401).
5. Se válida, gera access token (`GerarAccessToken`) e refresh token (`GerarRefreshTokenAsync`, persiste hash em `RefreshTokens`).
6. Retorna `AuthResponseDto` ao cliente.

3.3 Cadastro de aluno

Arquivos: `Controllers/AlunosController.cs:48-58`, `Services/AlunoService.cs:91-126`, `Services/MatriculaService.cs` (fluxo 3.4 encadeado).

1. Cliente → `POST /api/v1/Alunos/cadastro-completo` com `CadastroAlunoDto`.
2. `AlunoService.CadastrarAlunoCompletoAsync` valida e-mail/CPF únicos.
3. Abre transação de banco.
4. Cria `Aluno` (herda de `Usuario`) com senha já em hash.
5. Chama `MatriculaService.MatricularComAprovacaoAutomaticaAsync` (ver 3.4) de dentro da mesma transação.
6. Comita a transação; retorna mensagem de sucesso (varia se curso é pago ou gratuito).

3.4 Matrícula

Arquivos: `Controllers/MatriculasController.cs:74-84`, `Services/MatriculaService.cs:49-90,171-182,350-399`.

1. Cliente → `POST /api/v1/Matriculas` (ou chamada interna do fluxo 3.3).
2. `MatriculaService.MatricularComAprovacaoAutomaticaAsync` chama `ICursoService.GarantirCursoAbertoAsync` (valida que o curso aceita matrícula).
3. Cria `Matricula` com `Status = Pendente` (sem vincular Turma).
4. Verifica duplicidade (matrícula Aprovada já existente no mesmo curso).
5. Chama `matricula.Aprovar()` (muda status, não toca em Turma).
6. Se o curso é pago, cria `Pagamento` pendente.
7. Salva com proteção de concorrência (índice único do banco).
8. Retorna `MatriculaResponseDto`.

3.5 Início e conclusão de Quiz

Arquivos: `Controllers/AvaliacoesController.cs:130-142,236-253`, `Services/AvaliacaoService.cs:565-692`.

1. Cliente → `GET /api/v1/Avaliacoes/{id}/aluno/questoes` — retorna as questões sem checar limite de tentativas.
2. Aluno responde; cliente → `POST /api/v1/Avaliacoes/{id}/aluno/respostas` com as respostas.
3. `AvaliacaoService.EnviarRespostasAlunoAsync` valida janela de disponibilidade e completude das respostas.
4. Valida limite de tentativas (`tentativasRealizadas >= TentativasPermitidas`) — ponto único de bloqueio.
5. Cria `TentativaAvaliacao` + `RespostaAluno` para cada questão; corrige automaticamente questões objetivas.
6. Se `TipoAvaliacao == Quiz`: chama `ProgressoAlunoService.RecalcularProgressoQuizAsync`.
7. Retorna o resultado da tentativa ao cliente.

3.6 Consulta da revisão de Quiz

Arquivos: `Controllers/AvaliacoesController.cs:255-272`, `Services/AvaliacaoService.cs:694-719`.

1. Cliente → `GET /api/v1/Avaliacoes/{id}/aluno/tentativas/{tentativaId}/revisao`.
2. `AvaliacaoService.ObterRevisaoTentativaAsync` carrega a tentativa com Avaliação, Matrícula e Respostas (com questões/alternativas relacionadas).
3. Valida posse (a tentativa pertence ao aluno autenticado).
4. Valida que a tentativa já foi corrigida.
5. Retorna `RevisaoTentativaResponseDto` — **sem** nenhuma validação de limite de tentativas restantes (diferença estrutural frente ao fluxo 3.5).

Nota para quem for desenhar: este é o ponto de maior interesse para quem investiga o comportamento de "Revisão Detalhada" relatado como inconsistente no frontend — o diagrama de sequência deste fluxo, desenhado lado a lado com o do fluxo 3.5, evidencia visualmente que são caminhos de código completamente separados no backend.

3.7 Realização de Avaliação formal (Prova/Exercício)

Arquivos: os mesmos do fluxo 3.5 — `AvaliacaoService.cs:672-689` é onde a ramificação por `TipoAvaliacao` acontece.

1. Passos 1–5 idênticos ao fluxo 3.5.
2. Se `TipoAvaliacao != Quiz`: chama `ProgressoAlunoService.RecalcularNotaAvaliacaoAsync` (em vez de `RecalcularProgressoQuizAsync`), condicionado a não haver questão dissertativa pendente de correção manual.
3. Dispara notificação `TipoNotificacao.AvaliacaoCorrigida` ao aluno (não acontece no Quiz).

3.8 Consulta de resultados e certificados

Arquivos: `Controllers/CertificadosController.cs:21-33`, `Services/CertificadoService.cs:19-86`.

1. Cliente → `POST /api/v1/Certificados/matricula/{matriculaId}/emitir`.
2. `CertificadoService.EmitirCertificadoAsync` valida, em ordem: posse da matrícula → status Aprovada → pagamento confirmado (se pago) → progresso 100% → (só para Prova/Exercício) nenhuma avaliação pendente de lançamento de nota → (só para curso Acompanhado) curso encerrado.
3. Se tudo passar, marca `CertificadoEmitidoEm` (idempotente) e retorna `CertificadoResponseDto`.

4. Guia para o diagrama de casos de uso

4.1 Como os atores foram identificados

Os 4 papéis autenticados (Aluno, Professor, Coordenador, Admin) vêm diretamente dos 4 tipos concretos de `Usuario` (TPT) e são confirmados pelos atributos `[Authorize(Roles="...")]` em cada Controller (ver Manual Técnico, seção 7). O ator "Visitante" cobre as rotas `[AllowAnonymous]` e as duas rotas públicas de `CursosController` sem atributo de autorização.

4.2 Casos de uso por ator — informações textuais para montar o diagrama

Para cada caso de uso abaixo, construa na ferramenta externa um elemento "Caso de Uso" ligado ao(s) ator(es) indicado(s). As caixas a seguir trazem as informações textuais disponíveis extraídas do código (endpoint real, controller/método) — preencha pré-condições/fluxo alternativo detalhado olhando o Service correspondente, se a ferramenta exigir essa granularidade.

UC — Matricular-se em curso

Ator principal
Aluno (ou Visitante, no cadastro público)

Pré-condição
Curso existe e aceita matrícula (não é um curso Acompanhado encerrado)

Fluxo principal
Ver fluxo de sequência 3.4

Relação
`«include»` "Efetuar pagamento" — só se `Curso.Preco > 0` (confirmado em `MatriculaService.CriarPagamentoPendenteSeNecessarioAsync`)

Endpoint/Classe
`POST /Matriculas`, `POST /Alunos/cadastro-completo` — `MatriculaService`

UC — Responder Quiz / Avaliação formal

Ator principal
Aluno

Pré-condição
Avaliação publicada, dentro da janela de disponibilidade, limite de tentativas não atingido

Fluxo principal
Ver fluxos de sequência 3.5 e 3.7

Relação
`«include»` "Iniciar tentativa" (implícito no envio de respostas, não é endpoint separado)

Endpoint/Classe
`GET .../aluno/questoes`, `POST .../aluno/respostas` — `AvaliacaoService`

UC — Consultar revisão de Quiz concluído

Ator principal
Aluno

Pré-condição
Existe tentativa corrigida pertencente ao aluno

Fluxo principal
Ver fluxo de sequência 3.6

Observação
Caso de uso distinto de "Responder Quiz" — endpoint e método de Service próprios, sem relação `«include»` / `«extend»` com ele no backend

Endpoint/Classe
`GET .../tentativas/{id}/revisao` — `AvaliacaoService.ObterRevisaoTentativaAsync`

UC — Emitir certificado de conclusão

Ator principal
Aluno

Pré-condição
Ver cadeia completa de validações no fluxo de sequência 3.8

Endpoint/Classe

`POST /Certificados/matricula/{id}/emitir` — `CertificadoService`

UC — Verificar autenticidade de certificado

Ator principal
Visitante (não autenticado)

Endpoint/Classe
`GET /Certificados/verificar/{codigo}` — único endpoint de certificado sem exigência de login

UC — Gerenciar banco de questões e publicar avaliação/quiz

Ator principal
Professor

Relação
`«include»` "Gerar questões por IA" (opcional — dois endpoints dedicados, rate-limited) — não obrigatório para publicar uma avaliação, é um caminho alternativo de criação de questão

Endpoint/Classe
`POST /Avaliacoes`, `/{id}/questoes`, `/{id}/questoes/lote`, `/{id}/ia/gerar-questoes(-de-conteudo)` — `AvaliacaoService`

UC — Vincular/desvincular professor a curso

Ator principal
Admin, Coordenador

Endpoint/Classe
`POST` / `DELETE /Cursos/{id}/professores/{professorId}` — `CursoService`

Observação
Substitui, desde out/2026, a atribuição de professor via Turma como mecanismo real de "quem leciona o quê"

UC — Concluir / Encerrar / Reabrir curso (modelo Acompanhado)

Ator principal
"Concluir" → Professor; "Encerrar"/"Reabrir" → **Coordenador** (não Admin — confirmado por atributo de autorização específico)

Pré-condição de Encerrar
Professor já marcou conclusão

Aplica-se a
Só cursos com `ModeloCurso = Acompanhado`

Endpoint/Classe
`POST /Cursos/{id}/concluir`, `/encerrar`, `/reabrir` — `CursoService`

UC — Aprovar / rejeitar matrícula pendente

Ator principal
Admin, Coordenador

Endpoint/Classe
`PUT /Matriculas/{id}/aprovar`, `/rejeitar`, `/aprovar-lote` — `MatriculaService`

Observação
Na prática a maioria das matrículas já é aprovada automaticamente no momento da solicitação (ver fluxo 3.4) — este caso de uso cobre o restante/exceções e o fluxo em lote

4.3 Lista completa de casos de uso por ator (resumo)

Ator	Casos de uso confirmados por autorização real no código
Aluno	Login/Logout, recuperar senha, consultar catálogo, matricular-se, efetuar pagamento (simulado), acompanhar progresso, acessar conteúdo, responder Quiz/Avaliação, consultar revisão de Quiz, consultar notas/feedback, emitir certificado, consultar notificações, editar perfil
Professor	Login/Logout, gerenciar banco de questões (+ anexos), criar/publicar conteúdo didático, criar avaliação/quiz (+ gerar questões por IA), corrigir dissertativa, lançar nota, registrar feedback acadêmico, visualizar desempenho de turmas/cursos, marcar curso Acompanhado como concluído

Coordenador	Login/Logout, gerenciar cursos/módulos/turmas, vincular/desvincular professor a curso, aprovar/rejeitar matrícula, encerrar/reabrir curso Acompanhado, visualizar desempenho e taxa de adesão (escopo próprio), exportar relatório
Admin	Tudo que o Coordenador faz (exceto encerrar/reabrir curso Acompanhado) + gerenciar Coordenadores, definir imagem de capa de curso, listar todos os usuários, atribuir coordenador a curso
Visitante	Cadastrar-se (+ matrícula automática), login, recuperar senha, consultar catálogo público de cursos, verificar autenticidade de certificado

4.4 Orientação sobre `«include»` / `«extend»`

Use `«include»` apenas quando o sub-caso de uso é **sempre** necessário para completar o caso principal (ex.: matricular-se em curso pago sempre envolve criar uma cobrança pendente). Use `«extend»` quando o sub-caso é **opcional/condicional** (ex.: gerar questões por IA é um caminho alternativo, não obrigatório, de alimentar o banco de questões antes de publicar uma avaliação). Não presuma essas relações sem confirmar a condicionalidade real no Service correspondente.

4.5 Funcionalidade de interface vs. endpoint de API

Nem toda tela do frontend corresponde a um caso de uso novo — várias telas (Web/Mobile/Desktop) consomem os mesmos endpoints listados acima. Ao modelar, baseie os casos de uso nos **endpoints/Services confirmados** (lista acima), não em cada tela individual do frontend, para evitar duplicar o mesmo caso de uso sob nomes diferentes por plataforma.

5. Outros diagramas a avaliar

5.1 Diagrama de componentes (recomendado para esta Etapa)

Útil porque a Etapa 6 pede explicitamente "alinhamento com Web e Mobile". Componentes a representar: Web (React/Vite), Mobile (Expo), Desktop (Electron), Gateway (Node/Express, proxy puro), API .NET (Controllers→Services→Repositories), SQL Server. Conectores rotulados com o protocolo (HTTP/JSON, Bearer JWT). **Consultar:** `CLAUDE.md` (seção "Frontend architecture"/"Gateway"), `desktop/main.js`, `gateway/` (arquivo de entrada do proxy), `Program.cs` (CORS/pipeline).

5.2 Diagrama de implantação (opcional)

Útil se a apresentação for cobrar também a Etapa 8 (nuvem/DevOps). Representar: App Service do Azure (API + frontend estático), Azure SQL Database, Azure Blob Storage (uploads em produção), GitHub Actions (CI/deploy). **Consultar:** arquivo de workflow em `.github/workflows/`, seção `AzureStorage` de `appsettings.example.json`, `Program.cs:108-135` (registro condicional de Blob Storage).

5.3 Diagrama de atividades (opcional)

Mais útil para um fluxo com decisões visíveis, como a aprovação de matrícula (gratuito vs. pago) ou a emissão de certificado (cadeia de 4-6 validações sequenciais do fluxo 3.8) — um diagrama de atividades comunica melhor ramificações condicionais do que um de sequência. **Consultar:** mesmos arquivos do fluxo de sequência 3.4 ou 3.8.

5.4 Diagrama de estados (opcional, baixa prioridade)

Candidato natural: ciclo de vida de `Matricula` (`StatusMatricula`: Pendente → Aprovada/Rejeitada → Cancelada → Reaberta) ou de `TentativaAvaliacao` (`EmAndamento → Enviada → Corrigida`, com `Expirada` como estado alternativo). **Consultar:** `Models/StatusMatricula.cs`, métodos `Aprovar/Rejeitar/Cancelar/Reabrir` em `Matricula.cs`; `Models/PedagogicalEnums.cs` (`StatusTentativaAvaliacao`) e métodos `Iniciar/MarcarEnvio/MarcarCorrecao/Expirar` em `TentativaAvaliacao.cs`.

6. Matriz de rastreabilidade

Tabela de referência para localizar rapidamente a informação necessária de cada diagrama. Preenchida só com dados confirmados nesta auditoria — pendências marcadas explicitamente em vez de preenchidas por suposição.

Diagrama a elaborar	Elementos a investigar	Arquivos de origem	Classes/métodos/rotas	Pendências de validação
Classes — Usuários	Herança TPT, encapsulamento	Models/Usuario.cs, Aluno.cs, Professor.cs, Coordenador.cs, Admin.cs; Data/Configurations/UsuariosConfiguration.cs	Usuario.ConfigurarAcesso	Nenhuma
Classes — Cursos	Curso, Módulo, Turma, Matrícula, ProfessorCurso, Pagamento	Models/Curso.cs, Modulo.cs, Turma.cs, Matricula.cs, ProfessorCurso.cs, Pagamento.cs; Data/Configurations/AcademicoConfiguration.cs	MatriculaService, CursoService	Como representar Turma (ativa vs. legada) — ver 1.4
Classes — Avaliação	Banco de questões vs. snapshot publicado, tentativa, resposta	Models/QuestaoBanco.cs, QuestaoPublicada.cs, Avaliacao.cs, TentativaAvaliacao.cs, RespostaAluno.cs, LancamentoNotaAluno.cs; Data/Configurations/AvaliacoesConfiguration.cs	AvaliacaoService	Nenhuma
Classes — Progresso	3 níveis de progresso persistido	Models/ProgressoConteudoAluno.cs, ProgressoModuloAluno.cs, ProgressoCursoAluno.cs; Data/Configurations/ProgressoConfiguration.cs	ProgressoAlunoService	Nenhuma
Sequência — Login	Emissão JWT + refresh	Controllers/AuthController.cs, Services/AuthService.cs	AuthService.LoginAsync, GerarAccessToken	Nenhuma
Sequência — Matrícula	Aprovação automática sem Turma	Controllers/MatriculasController.cs, Services/MatriculaService.cs	MatricularComAprovacaoAutomaticaAsync	Nenhuma
Sequência — Quiz (responder)	Checagem de limite de tentativas	Controllers/AvaliacoesController.cs, Services/AvaliacaoService.cs	EnviarRespostasAlunoAsync	Nenhuma
Sequência — Quiz (revisão)	Endpoint separado, sem checagem de tentativas	idem acima	ObterRevisaoTentativaAsync	Comportamento do frontend ao chamar este endpoint não auditado (fora de escopo deste pacote)
Sequência — Certificado	Cadeia de pré-condições	Controllers/CertificadosController.cs, Services/CertificadoService.cs	EmitirCertificadoAsync	Nenhuma
Casos de uso — todos os atores	Autorização real por endpoint	Todos os 15 arquivos de Controllers/	Atributos [Authorize(Roles=...)] / [AllowAnonymous]	Nenhuma
Componentes	Web/Mobile/Desktop/Gateway/API/Banco	CLAUDE.md, desktop/main.js, gateway/, Program.cs	—	Arquivo de entrada exato do gateway não relido nesta sessão — confirmar nome do arquivo antes de desenhar
Implantação (opcional)	Azure App Service, SQL, Blob, CI/CD	.github/workflows/, appsettings.example.json	—	Nome exato do workflow YAML não confirmado nesta auditoria

7. Procedimento de execução pelo desenvolvedor

- ☐ Ler o *Manual_Tecnico_CSharp_Desktop.pdf* (Documento A deste pacote) por inteiro antes de abrir a ferramenta de modelagem.
- ☐ Abrir a solução (`Sistema Academico Integrado.sln`) e localizar, um a um, os arquivos indicados na matriz de rastreabilidade (seção 6).
- ☐ Conferir cada classe e cada fluxo diretamente no código-fonte atual — o código pode ter mudado entre a data deste manual (23/10/2026) e a data em que os diagramas forem efetivamente desenhados.
- ☐ Escolher a ferramenta externa de modelagem UML (Astah, Visual Paradigm, StarUML, Enterprise Architect ou diagrams.net).
- ☐ Criar os diagramas seguindo as instruções das seções 2 a 5 deste manual, um módulo/fluxo por vez.
- ☐ Comparar cada elemento desenhado com a implementação real (checklist da seção 1.5).
- ☐ Registrar divergências e dúvidas encontradas durante a modelagem (uma lista separada, fora deste manual).
- ☐ Validar a consistência entre os diagramas (ex.: toda classe que aparece numa mensagem de um diagrama de sequência deve existir, com o mesmo nome, no diagrama de classes correspondente).
- ☐ Exportar os diagramas conforme os requisitos do projeto acadêmico (formato exigido pela disciplina — confirmar com o enunciado da Etapa 6/Manual do PIM IV).

> **Responsabilidade:** a construção, edição e exportação dos diagramas na plataforma escolhida é inteiramente do desenvolvedor que executar este procedimento — não faz parte da elaboração destes dois manuais.

Glossário de notação UML (textual, sem exemplo visual)

Generalização (herança)
Seta com ponta triangular vazada, apontando da subclasse para a superclasse. Usar para a hierarquia `Usuario` (seção 2.5).

Associação
Linha simples entre duas classes, com multiplicidade anotada em cada extremidade (ex.: `1`, `0..1`, `0..*`, `1..*`).

Composição
Associação forte, representada com um losango preenchido do lado do "todo" — o "filho" não existe sem o "pai" (ex.: `Modulo` não existe sem `Curso`, confirmado pelo `DeleteBehavior.Cascade`).

Agregação
Associação mais fraca, losango vazado — o "filho" pode existir independentemente do "pai" (menos comum neste projeto que composição).

Realização
Seta tracejada com ponta triangular vazada, de uma classe concreta para a interface que ela implementa (ex.: `CursoService` → `ICursoService`).

Dependência
Seta tracejada simples — uma classe usa outra sem guardar referência permanente.

`«include»`
Em diagrama de casos de uso: o caso base sempre aciona o caso incluído como parte do seu fluxo.

`«extend»`
Em diagrama de casos de uso: o caso estendido só é acionado condicionalmente.

Ator
Figura "boneco" representando um papel externo que interage com o sistema (aqui: Aluno, Professor, Coordenador, Admin, Visitante).

Mensagem síncrona (diagrama de sequência)
Seta sólida com ponta cheia — chamada que espera retorno (padrão em chamadas HTTP request/response e chamadas de método `async/await` deste projeto).

Lifeline
Linha vertical tracejada abaixo de cada participante num diagrama de sequência, representando sua existência ao longo do tempo.